[Optional quote here.]

Abstract

[Abstract to be written.]

Keywords: requirements engineering, ambiguity, BERT, cloud computing, large language models, natural language processing.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The classification of cloud computing requirements from natural language documents is a well-studied problem in requirements engineering. Casalicchio and Cotumaccio's AI-CRAS framework [3] addressed it by fine-tuning a BERT model on a domain-specific dataset of cloud requirement sentences, achieving a recall of 0.96 and an F1-score of 0.92 on the binary task of identifying whether a sentence constitutes a requirement.

What AI-CRAS does not examine is the effect of *ambiguity*. Many requirements are phrased in ways that allow more than one interpretation — due to vague wording, grammatical structure, or missing context. Natural language requirements can exhibit several distinct forms of ambiguity: lexical, syntactic, semantic, pragmatic, and language-error (described in detail in Chapter 3, following the taxonomy of [1]). These are common in real-world requirement documents, but their effect on model performance has not been studied.

This thesis tests whether ambiguity matters for classification — and whether certain categories are more vulnerable to it than others. Using the same AI-CRAS dataset, each sentence is manually annotated with ambiguity labels. To answer RQ1, a BERT classifier is trained on the full dataset (baseline) and on the unambiguous subset and the two are compared. To answer RQ2, the same classifier is re-run on an LLM-deambiguified version of the dataset. To answer RQ3, an LLM is prompted to classify the sentences directly, in both zero-shot and few-shot configurations. For each experiment, results are broken down by the six cloud service categories to assess whether ambiguity affects them uniformly.

The three research questions are:

- **RQ1:** Does ambiguity negatively affect BERT classification performance, and which of the six cloud service categories is most affected?
- **RQ2:** Can LLM-based rewriting of ambiguous sentences improve BERT's accuracy — both overall and for the most ambiguity-sensitive categories?
- **RQ3:** Can an LLM used directly as a classifier match or exceed BERT?

The main contributions of this thesis are: an ambiguity-annotated extension of the AI-CRAS dataset; an empirical analysis of how ambiguity affects BERT-based classification, including per-category breakdowns; and a comparative evaluation of LLMs as both a disambiguation tool and a direct classifier.

Chapter 2

Literature Review

2.1 The AI-CRAS Baseline: Cloud Requirement Classification

The AI-CRAS framework [3] proposes an AI-driven methodology for automating the analysis and specification of cloud computing requirements expressed in natural language. It is the central reference point for this thesis: its ontology, dataset, model architecture, and hyperparameter configuration are adopted directly, and its published results serve as the benchmark against which all experiments in this work are measured.

2.1.1 Ontology

AI-CRAS defines a hierarchical cloud service ontology that organises requirements into three levels of granularity. This thesis uses the same ontology, unchanged, to classify every sentence in the dataset. The six main categories are:

- **Compute** — processing resources: virtual machines, containers, auto-scaling, load balancing.
- **Data Handling** — data storage and management: databases (SQL, NoSQL), storage systems, caching and data optimisation.
- **Network** — connectivity and traffic management: firewalls, load balancers, VPNs, DNS, latency and bandwidth specifications.
- **Security & Compliance** — protection and regulatory adherence: encryption, identity management, certifications (ISO 27001, GDPR), access control.
- **Management & Monitoring** — operational oversight: dashboards, logging, alerting, cost management, auditing.
- **Cloud Service Essentials** — foundational cloud properties: high availability, disaster recovery, multi-region deployment, service-level agreements.

Each main category contains three to five subcategories, and the lowest level defines 68 specific cloud features (e.g., key management, container orchestration, geo-replication). A sentence may carry multiple labels — this is a multi-label classification problem, and the six categories are treated as independent binary targets.

2.1.2 Dataset

The experiments in this thesis use the exact same dataset constructed by AI-CRAS. It consists of 2,164 sentences extracted from public Request for Proposal (RFP) documents, supplemented with seven synthetically generated documents produced via GPT-4 to increase domain coverage. Each sentence was independently labelled by two annotators along two dimensions: a binary indicator distinguishing requirements from non-requirements (68.1% are requirements), and one-hot encoded assignments to the six ontology categories. Disagreements between annotators led to exclusion of the disputed sample, preserving consistency.

2.1.3 Model and Configuration

AI-CRAS's BERT-based classifier is the starting point for the pipeline used in this thesis. The architecture and hyperparameters are reproduced exactly:

- **Model**: $\text{BERT}_{\text{BASE-UNCASED}}$ (12 layers, 768 hidden units, 12 attention heads), augmented with a dropout layer (rate 0.3) and a linear output layer mapping the pooled representation to six logits.
- **Loss**: binary cross-entropy with logits, treating each of the six categories as an independent binary target (multi-label setting).
- **Optimiser**: Adam, with learning rate 3×10^{-5} and weight decay 10^{-6}.
- **Preprocessing**: WordPiece tokenizer in uncased mode, maximum sequence length of 300 tokens.
- **Training**: 85% training / 15% validation split (seed 42), batch size of 16, up to 5 epochs with early stopping on validation loss.
- **Inference threshold**: 0.20, chosen in the original work to maximise recall without excessive precision loss.

A full description of the training procedure and evaluation protocol is given in Chapter 4.

2.1.4 AI-CRAS Results as Benchmark

On the original dataset, AI-CRAS reported the following results, which serve as the performance ceiling for the experiments in this thesis:

- **Binary classification** (requirement *vs.* non-requirement): recall of 0.96, precision of 0.88, F1-score of 0.92.
- **Multi-label classification**: macro-averaged F1-score of 0.76, with per-category F1 scores ranging from 0.70 (Compute) to 0.84 (Security & Compliance).
- **ROC-AUC**: 0.96 for both micro and macro averages.

These numbers are the baseline. Any degradation observed when ambiguous sentences are present, and any recovery observed after LLM-based disambiguation, is measured relative to them. The framework, however, operates under an implicit assumption that its input sentences are clear and well-formed — an assumption that rarely holds in real-world procurement documents, where requirements are routinely vague, structurally ambiguous, or context-dependent. Whether such ambiguity degrades classification performance is the question that motivates the remainder of this review.

2.2 Ambiguity in Natural Language Requirements

Having established AI-CRAS as the baseline, the next question is: what form does the missing piece, ambiguity, take? Bano [1] defines ambiguity in requirements engineering as "having multiple interpretations despite the reader's knowledge of the RE context", and identifies it as more intractable than other defects such as incompleteness. Crucially, an ambiguous requirement can be correctly identified *as* a requirement by a classifier, while still being fundamentally misunderstood — which means that high classification scores on a benchmark do not guarantee that the input was well understood.

The taxonomy established by Berry and systematised by Bano [1] classifies ambiguity into five types. This is the taxonomy adopted in this thesis to annotate every sentence in the AI-CRAS dataset:

- **Lexical ambiguity**: a word or phrase has multiple meanings (e.g., "high availability" may imply different uptime guarantees to different stakeholders).
- **Syntactic ambiguity**: the grammatical structure admits more than one valid parse (e.g., a modifier whose attachment is ambiguous between two clauses).
- **Semantic ambiguity**: the logical content supports multiple interpretations even when the syntax is clear (e.g., "ensure continuity of service at all times" without defining what counts as continuity).
- **Pragmatic ambiguity**: the meaning depends on implicit context not present in the text (e.g., "this will include system maintenance windows" without specifying what a maintenance window entails).
- **Language-error ambiguity**: grammatical mistakes or typographical errors obscure the intended meaning (e.g., "All the equipment's/Devices in the path have to be in HA mode" leaves the acronym undefined and contains a spurious apostrophe).

These five categories are not rare edge cases. Bano's systematic mapping of the RE literature found that 81% of empirical work on requirements ambiguity focuses on detection alone, with almost no attention to resolution. For this thesis, the implication is direct: the AI-CRAS baseline was trained and evaluated without regard for ambiguity, and the literature offers no precedent for whether resolving ambiguity, rather than merely detecting it, can improve downstream classification. The annotation procedure that operationalises this taxonomy on the AI-CRAS dataset, and the empirical distribution of each ambiguity type, are described in Chapter 3.

2.3 Classification and Disambiguation Models

To study the effect of ambiguity on cloud requirement classification, this thesis needs two things: a classifier to measure its effect, and a tool to attempt to repair it. BERT handles the first; large language models handle the second.

2.3.1 BERT: The Classifier Under Test

BERT [4] is built on the transformer architecture [6], which replaced recurrent networks with a self-attention mechanism that lets every token attend to all others simultaneously. BERT extends this by pre-training on large corpora with masked language modelling, producing representations that fine-tune well on downstream tasks with relatively little labelled data. Fine-tuned BERT classifiers have demonstrated strong performance across requirements engineering tasks, including identifying requirements from unstructured documents and classifying functional and non-functional requirements.

The reason BERT, specifically, is used in this thesis is methodological: it is the model family chosen by AI-CRAS, and reproducing the same architecture ensures direct comparability with the published baseline. Any difference in performance between the original dataset and the ambiguity-filtered or deambiguified variants can then be attributed to the data, not to a change in model.

At the same time, the rapid evolution of large language models in recent years raises the question of whether a general-purpose model can bypass the fine-tuning step altogether and perform the same classification task directly via prompting. This alternative is explored in Section 2.3.2.

2.3.2 Large Language Models: Disambiguation and Direct Classification

If BERT measures the damage that ambiguity causes, large language models offer two strategies for repairing it.

The first is disambiguation. The GPT family [5] shifted the paradigm from fine-tuning on labelled data to prompting with natural language instructions. This makes LLMs natural candidates for rewriting: given an ambiguous sentence, the model can be asked to produce a clearer version that preserves the original intent, without any task-specific training. This thesis tests whether feeding such rewritten sentences back to the BERT classifier recovers the performance lost to ambiguity (RQ2).

The second is removing BERT from the pipeline entirely. Brown et al. demonstrated that GPT-3 can perform few-shot classification by conditioning on a handful of labelled examples in the prompt [2], and Wei et al. showed that instruction-tuned models generalise to unseen tasks in zero-shot settings [7]. Whether this capability extends to the technical domain of cloud requirement classification — and whether it can match a fine-tuned BERT — is what RQ3 investigates.

Chapter 3

Dataset and Annotation

This chapter describes the AI-CRAS dataset that forms the basis of all experiments, the ambiguity annotations added to it, and the statistical properties that motivate the experimental design.

3.1 The AI-CRAS Dataset

The experiments use the dataset introduced by Casalicchio and Cotumaccio [3]. It consists of 2,267 sentences extracted from publicly available Request for Proposal (RFP) documents related to cloud procurement and supplemented with seven synthetically generated documents produced via GPT-4 to increase domain coverage. Each sentence was independently examined by two annotators and classified along two dimensions.

First, a binary label indicates whether the sentence constitutes a cloud requirement (`goal = 1`) or not (`goal = 0`). Second, a set of six one-hot encoded indicators assigns the sentence to one or more cloud service categories, following the AI-CRAS ontology described in Chapter 2: `compute`, `data_handling`, `network`, `security_compliance`, `management_monitoring`, and `cloud_service_essentials`. The assignment is multi-label: a single sentence can carry zero, one, or several categories. Disagreements between the two annotators led to exclusion of the disputed sample, preserving consistency.

3.2 Ambiguity Annotation

To support the experiments, each sentence in the dataset was manually annotated for ambiguity. The annotation follows the five-way taxonomy established by Berry and systematised by Bano [1], introduced in Chapter 2. Two columns were added:

- `ambuiguity`: binary, set to 1 if the sentence exhibits any form of ambiguity and 0 otherwise.
- `ambiguity_type`: integer encoding the specific type, with values 1–5 corresponding to lexical, syntactic, semantic, language-error, and pragmatic ambiguity respectively. A value of 0 indicates no ambiguity.

The annotation was conducted in a single pass: each sentence received exactly one type label, corresponding to the dominant form of ambiguity present. The five types are defined as follows, with representative examples drawn from the dataset:

Lexical ambiguity (type 1). A word or phrase conveys multiple meanings depending on context. *Example:* "The database server storage has to be provided on high speed disks (SSD's) for better performance" — "high speed" and "better performance" are qualitative terms that admit no single quantitative interpretation. In procurement documents, lexical ambiguity is pervasive: terms such as "adequate", "high availability", "fast response", and "scalable" recur without numerical bounds.

Syntactic ambiguity (type 2). The grammatical structure admits more than one valid parse. *Example:* "CSP / Bidder to conduct vulnerability assessment scanning of the Portal/Website for every 1 hour till the website is LIVE for rendering services" — the modifier "for every 1 hour" could attach to "scanning" or to "LIVE", yielding different obligations.

Semantic ambiguity (type 3). The logical content supports multiple interpretations even when the syntax is clear. *Example:* "It is the prime responsibility of CSP to ensure continuity of service at all times of the Agreement including exit management period (may be one month)" — the phrase "at all times" conflicts with the parenthetical "may be one month" without clarifying which constraint takes precedence.

Language-error ambiguity (type 4). Grammatical mistakes or typographical errors obscure the intended meaning. *Example:* "All the equipment's/Devices in the path have to be in HA mode" — the spurious apostrophe in "equipment's" suggests a typo rather than a possessive, and the acronym "HA" is left undefined, making the specification ambiguous.

Pragmatic ambiguity (type 5). The meaning depends on implicit context not present in the text. *Example:* "This will include system maintenance windows" — without a definition of what constitutes a maintenance window (duration, frequency, permitted downtime), the requirement is underspecified and cannot be verified.

Each example illustrates the connection between the abstract taxonomy and the concrete challenges of the dataset: the ambiguity is not theoretical — it arises from the way requirements are actually written in procurement documents, and it directly affects whether a classifier can reliably determine what the sentence is asking for.

3.3 Dataset Statistics

Of the 2,267 sentences, 689 (30.4%) are annotated as ambiguous. The breakdown by ambiguity type is shown in Table 3.1.

Lexical ambiguity dominates, accounting for over four-fifths of all ambiguous sentences. This is a direct consequence of RFP language: qualitative terms such as "high speed", "adequate load balancers", and "better performance" are routinely used without quantitative specification. Syntactic ambiguity is the rarest, consistent with the formal, template-driven style typical of procurement documents.

Ambiguity is not uniformly distributed across the six categories. Figure 3.1 visualises the breakdown, and Table 3.2 reports the per-category rates.

Cloud Service Essentials and Data Handling exhibit the highest ambiguity rates (43.9% and 39.8%), suggesting that requirements related to foundational cloud properties and data management

Table 3.1: Distribution of ambiguity types.

Ambiguity Type	Count	% of Ambiguous
Lexical	581	84.3%
Pragmatic	61	8.9%
Semantic	28	4.1%
Language-error	17	2.5%
Syntactic	2	0.3%
Total ambiguous	689	100%

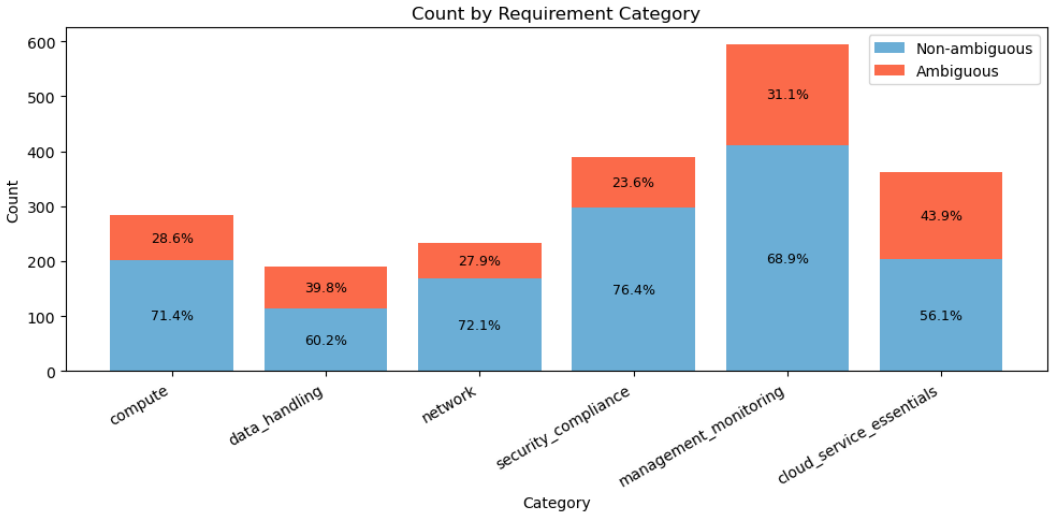

Figure 3.1: Stacked breakdown of ambiguous and non-ambiguous sentences per cloud service category.

are particularly prone to vague and underspecified phrasing. Security & Compliance requirements are the least ambiguous (23.6%), likely because security-related clauses in procurement documents tend to reference specific standards and certifications, which reduces vagueness.

The LLM-deambiguified variant of this dataset — in which every ambiguous sentence is rewritten with concrete clarifications — is generated from the annotated corpus using the procedure described in Chapter 4. It forms a parallel dataset that enables direct comparison of BERT's performance on original and disambiguated inputs.

Table 3.2: Proportion of ambiguous sentences per category.

Category	Ambiguous	Total	Rate
Cloud Service Essentials	159	362	43.9%
Data Handling	76	191	39.8%
Management & Monitoring	185	595	31.1%
Compute	81	283	28.6%
Network	65	233	27.9%
Security & Compliance	92	390	23.6%

Chapter 4

Methodology

This chapter describes the three components of the experimental pipeline: the BERT-based classifier that serves as the primary model under test, the LLM-based disambiguation procedure used to produce clarified variants of ambiguous sentences, and the LLM direct classification setup—spanning three models in both zero-shot and few-shot configurations—evaluated as an alternative to BERT. The chapter closes with the full experimental design, defining the dataset subsets and model configurations on which each component is evaluated.

4.1 BERT Classification Pipeline

The classifier follows the architecture and configuration established by the AI-CRAS framework [3], adopted unchanged to ensure direct comparability of results.

4.1.1 Model Architecture

The model is built on $\text{BERT}_{\text{BASE-UNCASED}}$ [4], a 12-layer transformer encoder with 768-dimensional hidden representations and 12 attention heads, pre-trained on English text with a vocabulary of 30,522 WordPiece tokens. The pre-trained encoder is augmented with two task-specific layers: a dropout layer with rate 0.3 for regularisation, and a linear layer that maps the pooled 768-dimensional output of the `[CLS]` token to six logits, one for each cloud service category. The six categories are treated as independent binary classification targets (multi-label), so the model can assign zero, one, or several labels to any given sentence.

The loss function is binary cross-entropy with logits (`BCEWithLogitsLoss`), which combines a sigmoid activation with binary cross-entropy in a numerically stable single operation. Training uses the Adam optimiser with a learning rate of 3×10^{-5} and weight decay of 10^{-6}. No learning rate scheduling is applied.

4.1.2 Tokenization and Preprocessing

Input sentences are tokenised using the BERT WordPiece tokenizer in its uncased variant, converting all text to lowercase. Each sentence is padded or truncated to a fixed length of 300 tokens—the same limit used in AI-CRAS, sufficient to cover the longest sentences in the dataset. Special tokens `[CLS]` and `[SEP]` are prepended and appended respectively, and an attention mask distinguishes real tokens from padding.

4.1.3 Training Procedure

The dataset is split into training (85%) and validation (15%) partitions using stratified random sampling with seed 42, following the AI-CRAS protocol. Training proceeds for a maximum of 5 epochs with a batch size of 16, using full-batch validation after each epoch. An early stopping mechanism halts training if validation loss does not decrease between consecutive epochs, and the model checkpoint with the lowest validation loss is retained for evaluation.

All experiments are run on an Apple MPS (Metal Performance Shaders) GPU, with PyTorch as the deep learning framework. Both the training and the evaluation results are fully reproducible given the same seed and hardware.

4.1.4 Inference and Threshold

At inference time, the model outputs six logits per sentence, which are passed through a sigmoid function to obtain probabilities in $[0, 1]$. A sentence is considered to carry a given label if the corresponding probability exceeds the threshold of 0.20, the value used by AI-CRAS. This threshold was chosen in the original work to maximise recall without an excessive cost in precision, and is preserved here for consistency.

4.1.5 Evaluation Metrics

For the binary task (requirement *vs.* non-requirement), a sentence is classified as a requirement if it receives at least one label above the threshold. Standard precision, recall, and F1-score are reported.

For the multi-label task, the following metrics are computed:

- **Exact match accuracy**: the fraction of sentences for which the predicted label set exactly matches the ground truth.
- **Hamming loss**: the fraction of individual label predictions that are incorrect (lower is better).
- **Jaccard score**: the size of the intersection divided by the size of the union of predicted and true label sets, macro-averaged across labels.
- **Precision, recall, F1-score** at three aggregation levels:
  - *Micro-averaged*: contributions of all labels pooled together, giving equal weight to every prediction.
  - *Macro-averaged*: per-label scores averaged with equal weight, so that all six categories contribute equally regardless of frequency.
  - *Per-label*: a separate score for each of the six categories, enabling identification of which categories are most affected by ambiguity.
- **ROC curves and AUC**: receiver operating characteristic curves are plotted per label, and the area under the curve is computed per label, as well as for the micro and macro averages, providing a threshold-independent measure of discriminative ability.

4.2 LLM-Based Disambiguation

To test whether ambiguity can be mitigated by rewriting, an LLM-based disambiguation procedure was applied to the annotated dataset. The goal is to produce, for each ambiguous sentence, a version that preserves the original meaning while removing vagueness through the addition of concrete, measurable clarifications.

4.2.1 Model and Prompting Strategy

The disambiguation uses $\text{GPT}_{\text{5.4-mini}}$ (OpenAI), accessed via the chat completions API with temperature set to 0 to maximise determinism and reproducibility. The model receives a detailed system prompt that specifies:

- The task: for every row where the ambiguity flag is 1, rewrite the sentence into a new field; for rows where it is not, copy the original sentence verbatim with no changes.
- Core constraints: preserve the exact original meaning; only clarify ambiguity; add measurable values, technical specifics, or concrete constraints — preferably in parenthetical form.
- Allowed clarification patterns with realistic generic values (e.g., "high speed" → "high speed (e.g., 100 MB/s throughput)"; "secure" → "secure (e.g., AES-256 encryption at rest, TLS 1.3 in transit)").
- Strict anti-laziness rules: no vague phrases, every rewritten sentence must include a concrete clarification, and varied but realistic values should be used across rows.
- Output format: valid CSV with two columns — `original` and `deambiguified_description` — preserving row order and quoting every field.

The complete prompt is reproduced in Appendix A.

4.2.2 Batch Processing and Validation

The dataset is processed in chunks of 10 rows to keep each API call within context length limits while allowing the prompt to remain detailed. Each chunk is sent independently, and the returned CSV is validated before proceeding to the next chunk. Validation checks that: (i) the CSV is syntactically valid and contains exactly the two expected columns; (ii) the number of returned rows matches the number of input rows; and (iii) the original-text column preserves each input sentence exactly, in order. If any check fails, the chunk is retried up to three times with an auto-generated repair prompt that includes the validation error and the previous (invalid) output. After the first successful parse, an additional verification step compares the returned originals to the input originals; if they differ, a second repair cycle is triggered to fix the order or content.

Non-ambiguous sentences are retained verbatim. The result is a parallel dataset in which every row of the original corpus has a counterpart in the column `deambiguified_description`, permitting side-by-side comparison.

4.3 LLM as Direct Classifier

Beyond rewriting, LLMs can perform classification directly through prompting, bypassing the BERT pipeline entirely. This configuration is evaluated to determine whether a general-purpose model, instructed in natural language, can approach or exceed the performance of the domain-specific fine-tuned classifier.

Three models are evaluated to assess the relationship between model capability and classification performance:

- **GPT$_{\text{4.1-nano}}$**: a lightweight, low-cost model designed for simple tasks. It serves as a baseline for checking whether even the cheapest models can perform competent cloud-requirement classification.
- **GPT$_{\text{5.4-mini}}$**: a mid-range model offering a favourable balance of capability, speed, and cost. This is the primary model for both disambiguation (Section 4.2) and direct classification.
- **GPT$_{\text{5.5}}$**: a large, high-capability model optimised for complex reasoning. It probes the upper bound of LLM classification performance on this task.

All models are accessed via the OpenAI chat completions API with temperature set to 0, maximising determinism and reproducibility. No sampling or top-p filtering is applied.

4.3.1 Prompt Design

The classification prompt contains three components: a system-level instruction, the six category definitions, and the batch of sentences to classify. The system instruction directs the model to output valid JSON only, with no explanatory text. The category definitions—drawn from the AI-CRAS ontology and identical to those presented in Section 2.1—are included inline in every user message:

> **Compute:** Processing resources: virtual machines, container orchestration, serverless computing, CPU/GPU and memory specifications.
> **Data Handling:** Data storage and management: databases (SQL, NoSQL), storage systems, caching and data optimisation.
> **Network:** Connectivity and traffic management: firewalls, load balancers, VPNs, DNS, latency and bandwidth specifications.
> **Security & Compliance:** Protection and regulatory adherence: encryption, identity management, certifications (ISO 27001, GDPR), access control.
> **Management & Monitoring:** Operational oversight: dashboards, logging, alerting, cost management, auditing.
> **Cloud Service Essentials:** General cloud properties: elasticity, multitenancy, pay-per-use billing, availability zones, SLAs.

Each sentence may receive zero, one, or multiple labels. A sentence that receives at least one label is classified as a requirement; otherwise, it is a non-requirement. The prompt instructs the model to output a JSON array with one object per sentence, where each object contains:

- `id`: an integer matching the sentence's position in the batch;

- `labels`: a list of strings, each drawn from the six category names in their canonical form;
- `is_requirement`: a boolean indicating whether the sentence describes a requirement.

4.3.2 Zero-Shot Setting

In the zero-shot setting, the model receives only the sentence batch and the category definitions—no labelled examples are provided. This configuration tests the model's ability to perform the classification task purely from its pre-trained knowledge of cloud computing terminology, without any task-specific fine-tuning or demonstration. The model must map each sentence to the six categories using only the one-line ontology descriptions supplied in the prompt, and must determine the binary requirement status from the label set.

Zero-shot classification is the most parameter-efficient setup: it requires no labelled data, no training infrastructure, and a single API call per batch. If an off-the-shelf LLM can approach BERT-level performance in this configuration, it would represent a significant practical advantage for domains where labelled datasets are scarce or expensive to produce. Conversely, a large performance gap would indicate that the domain-specific knowledge captured by the fine-tuned BERT is not trivially replicable through general-purpose pre-training alone.

The zero-shot configuration also serves as a clean lower bound for the LLM conditions: any improvement observed in the few-shot setting can be attributed entirely to the addition of in-context examples, isolating the contribution of demonstration-based learning from the model's intrinsic knowledge.

4.3.3 Few-Shot Setting

In the few-shot setting, the prompt is augmented with a set of labelled demonstrations that show the model how to perform the classification task by example. The demonstrations are selected from a dedicated example pool—the first 200 rows of the dataset—which is held out from the evaluation set to prevent data leakage.

Example Selection

For each of the six categories, one sentence that carries that label, contains no ambiguity (`ambuiguity` = 0), and has exactly one label overall is selected as a clean prototype. The remaining slots per category are filled to match the underlying label-count distribution of the example pool: the proportion of single-label to multi-label sentences among the demonstrations is deliberately aligned with the proportion observed among labelled sentences in the pool (approximately 75% single-label, 25% multi-label). This ensures the model is exposed to both pure-category examples and legitimate label combinations in proportions consistent with the data, rather than being biased toward either extreme.

In addition, non-requirement sentences—those carrying zero labels—are included in proportion to their frequency in the pool (approximately 25% of demonstrations), providing explicit negative examples. The total number of demonstrations is determined automatically: 12 labelled examples (one prototype plus one distribution-matched example per category) plus non-requirement examples computed as $n_{\text{non-req}} = \lfloor 12 \cdot p_{\text{non-req}}/(1 - p_{\text{non-req}}) \rfloor$ where $p_{\text{non-req}}$ is the proportion of zero-label

sentences in the pool. For the pool used in this work (200 rows, first portion of the dataset), this yields 4 non-requirement demonstrations for a total of 16.

Sentences that carry multiple labels can serve as examples for more than one category. The selection algorithm iterates over categories, uses a with-replacement mechanism across categories but ensures no sentence is duplicated within the full demonstration set, and records the chosen sentences with their complete label vectors.

In-Context Learning Protocol

The demonstrations are formatted as sentence–label pairs and prepended to the batch instruction, maintaining the same JSON output format used in the zero-shot condition. The model is expected to generalise from these examples to unseen sentences, applying the category definitions with the benefit of having observed concrete instances of each class.

The few-shot configuration tests whether in-context learning—a capability for which LLMs are specifically known [2]—can compensate for the absence of supervised fine-tuning. If the few-shot performance significantly exceeds the zero-shot baseline, it suggests that the model benefits from concrete demonstrations of the classification task beyond what the category definitions alone convey. If the gain is marginal, it indicates that the category definitions are sufficiently informative on their own, or that the model's pre-training already provides adequate coverage of cloud-requirement language.

Design Considerations

The example pool is deliberately subsetted to 200 rows and excluded from the classification set. This ensures a clean separation: no sentence seen during in-context learning is ever evaluated. The remaining 1,966 sentences form the classification set, maintaining a large evaluation sample despite the held-out pool. The pool size is chosen to provide sufficient candidate diversity—each category must have at least one unambiguous single-label and one additional example—while keeping the hold-out loss minimal.

The selection strategy is distribution-matched: the proportion of single-label, multi-label, and non-requirement demonstrations reflects the corresponding proportions observed in the example pool. This prevents the demonstrations from introducing a label-count bias that might skew the model toward under- or over-assigning categories. All selected demonstrations are unambiguous (`ambuiguity` = 0), eliminating the risk of the model internalising unclear patterns from vague or underspecified examples.

The binary requirement flag is derived from the label set, consistent with the zero-shot protocol: a sentence receiving at least one label is a requirement. The demonstrations include both requirement and non-requirement examples, with non-requirement examples presented explicitly as sentences carrying an empty label set.

4.4 Experimental Design

The classifiers are evaluated on a family of dataset subsets designed to isolate the effect of ambiguity and to measure the contribution of disambiguation.

4.4.1 Baseline

The **baseline** subset consists of the full, case-insensitively deduplicated dataset comprising 2,166 sentences, containing both ambiguous (689) and non-ambiguous (1,578) examples. This reproduces the AI-CRAS configuration as faithfully as possible and serves as the reference against which all other conditions are compared.

4.4.2 No-Ambiguity Subset

A second subset, **no-ambiguity**, excludes all sentences flagged as ambiguous (`ambuiguity = 1`), leaving only the 1,578 unambiguous sentences. This tests the hypothesis that training and evaluating exclusively on clear sentences improves performance.

4.4.3 Deambiguified Subset

A third subset, **deambiguified**, is derived from the LLM-rewritten dataset (Section 4.2). Every ambiguous sentence is replaced by its LLM-rewritten counterpart, while non-ambiguous sentences are kept unchanged. This tests whether disambiguation improves classification performance.

4.4.4 LLM Direct Classification

The three LLMs ($GPT_{4.1\text{-nano}}$, $GPT_{5.4\text{-mini}}$, $GPT_{5.5}$) are evaluated as standalone classifiers on the full 2,166-sentence dataset, in both zero-shot and few-shot configurations. No training or fine-tuning is performed; classification relies entirely on the prompt and, in the few-shot case, on in-context demonstrations drawn from a 200-row example pool disjoint from the evaluated set (1,966 sentences). Evaluation uses the same multi-label and binary metrics as the BERT pipeline, excluding ROC-AUC.

4.4.5 Per-Category Analysis

Every experiment reports metrics not only globally but also broken down by the six cloud service categories. This per-category analysis is motivated by the finding in Chapter 3 that ambiguity rates vary substantially across categories (from 23.6% for Security & Compliance to 43.9% for Cloud Service Essentials), and identifies whether ambiguity degrades performance uniformly or whether some categories are disproportionately affected—and whether LLMs exhibit the same category-level variation as the fine-tuned BERT model.

Table 4.1 summarises all experimental conditions.

Table 4.1: Overview of experimental conditions. Models in parentheses are variants of the same condition.

Condition	Dataset	Classifier
Baseline	Full original (2,166)	BERT
No ambiguity	Unambiguous only (1,578)	BERT
Deambiguified	All ambiguous rewritten (2,166)	BERT
Zero-shot	Full original (2,166)	$GPT_{4.1\text{-nano}}$ / $GPT_{5.4\text{-mini}}$ / $GPT_{5.5}$
Few-shot	Example pool held out (1,966)	$GPT_{4.1\text{-nano}}$ / $GPT_{5.4\text{-mini}}$ / $GPT_{5.5}$

All BERT experiments are run with the same hyperparameter configuration and the same random seed (42) to ensure that performance differences across conditions are attributable solely to the data, not to training stochasticity. All LLM experiments use temperature 0 and identical prompts (differing only in the presence or absence of few-shot demonstrations and the model identifier), so that differences across models and settings reflect genuine capability differences rather than prompt engineering artefacts.

Chapter 5

Experimental Evaluation

This chapter presents the results of the four experimental scenarios designed to answer the three research questions. Each experiment is described in terms of its setup, the results obtained, and a brief interpretation.

5.1 Experimental Setup

[To be completed: hardware, model versions (BERT variant, LLM), training hyperparameters, random seeds, dataset splits used across all experiments.]

5.2 Baseline: BERT on the Full Dataset

[To be completed: results of fine-tuned BERT on the complete original dataset, both binary classification and multi-label category classification. Establishes the performance reference for all subsequent comparisons.]

5.3 RQ1 — Effect of Ambiguity on BERT Classification

[To be completed: results of BERT trained and/or evaluated on the subset of unambiguous sentences. Comparison against baseline to assess whether removing ambiguous examples improves or changes performance. Breakdown by ambiguity type if applicable.]

5.4 RQ2 — BERT on LLM-Disambiguated Sentences

[To be completed: results of BERT evaluated on the version of the dataset where ambiguous sentences have been rewritten by the LLM. Comparison against baseline and RQ1 results.]

5.5 RQ3 — LLM as Direct Classifier

This section presents the results of using $\text{GPT}_{\text{5.4-mini}}$ as a standalone classifier for cloud requirement identification, bypassing the BERT pipeline entirely. The model is evaluated in both zero-shot and few-shot configurations on the full 2,166-sentence dataset. All experiments use temperature 0, the

same category definitions (Section 4.3.1), and the batch-processing protocol described in Section 4.3. The same set of multi-label metrics (exact-match accuracy, Hamming loss, Jaccard score, micro-, macro-, and per-label F1) and the binary requirement F1-score are reported.

5.5.1 Zero-Shot Results

In the zero-shot setting, $GPT_{5.4\text{-mini}}$ classifies 2,166 sentences using only the category definitions as guidance, with no labelled examples. The results are shown in Table 5.1.

Table 5.1: Zero-shot classification results of $GPT_{5.4\text{-mini}}$ on the full dataset (2,166 sentences).

Metric	Value
Exact Match Accuracy	0.407
Hamming Loss	0.148
Jaccard Score	0.410
Micro F1	0.579
Macro F1	0.574
Binary Requirement F1	0.840

The model achieves a binary requirement F1-score of 0.840, demonstrating reliable discrimination between requirements and non-requirements from the category descriptions alone. The macro-averaged F1 of 0.574 and micro-averaged F1 of 0.579 indicate moderate overall performance on the multi-label task, with an exact-match accuracy of 0.407—meaning that just over 40% of sentences receive exactly the correct label set.

Table 5.2 breaks down performance by cloud service category.

Table 5.2: Per-label zero-shot classification results of $GPT_{5.4\text{-mini}}$.

Category	Precision	Recall	F1
Compute	0.423	0.694	0.525
Data Handling	0.364	0.885	0.516
Network	0.561	0.849	0.675
Security & Compliance	0.599	0.869	0.709
Management & Monitoring	0.642	0.591	0.616
Cloud Service Essentials	0.374	0.431	0.401
Macro Average	0.494	0.720	0.574

Performance varies considerably across categories. Security & Compliance is classified most reliably (F1 = 0.709), followed by Network (0.675) and Management & Monitoring (0.616). Cloud Service Essentials is the hardest category (F1 = 0.401), with the lowest precision (0.374) and recall (0.431) among all six categories. The Recall column reveals a consistent pattern: the model is substantially more willing to assign a label than to withhold it, as reflected in recall scores that exceed precision across all categories. At the extreme, Data Handling achieves a recall of 0.885 but a precision of only 0.364, indicating that the model assigns this label to more than twice as many sentences as warranted by the ground truth. This over-prediction pattern—high recall, low precision—is systematic and is explored further in the few-shot results.

5.5.2 Few-Shot Results

In the few-shot setting, the prompt is augmented with 16 demonstration examples drawn from a 200-row example pool disjoint from the classification set (Section 4.3.3). The examples are distribution-matched: 4 non-requirement sentences, 9 single-label sentences, and 3 multi-label sentences, reflecting the label-count distribution of the pool. Table 5.3 summarises the overall multi-label metrics, and Table 5.4 provides the per-category breakdown.

Table 5.3: Few-shot classification results of $GPT_{5.4\text{-mini}}$ on 1,966 sentences (200 held out as example pool).

Metric	Value
Exact Match Accuracy	0.396
Hamming Loss	0.148
Jaccard Score	0.405
Micro F1	0.573
Macro F1	0.568
Binary Requirement F1	0.832

Table 5.4: Per-label few-shot classification results of $GPT_{5.4\text{-mini}}$.

Category	Precision	Recall	F1
Compute	0.478	0.673	0.559
Data Handling	0.382	0.836	0.525
Network	0.566	0.806	0.665
Security & Compliance	0.607	0.828	0.700
Management & Monitoring	0.551	0.659	0.600
Cloud Service Essentials	0.346	0.377	0.361
Macro Average	0.488	0.696	0.568

The few-shot results follow the same overall pattern as the zero-shot configuration: Security & Compliance remains the strongest category (F1 = 0.700), Cloud Service Essentials the weakest (F1 = 0.361), and the model continues to exhibit higher recall than precision across all categories. The distribution-matched demonstrations appear to have a modest regularising effect on two categories—Compute F1 improves from 0.525 to 0.559, and Data Handling from 0.516 to 0.525—while producing comparable or slightly lower scores on the remaining four categories. The macro-averaged F1 drops marginally from 0.574 (zero-shot) to 0.568 (few-shot), and the binary requirement F1 decreases from 0.840 to 0.832. The systematic over-prediction observed in zero-shot persists: the model's average recall across categories remains at 0.696, while average precision stays at 0.488.

The few-shot demonstrations were curated with considerable care—unambiguous sentences, distribution-matched label counts, and explicit non-requirement examples—yet they do not yield a consistent improvement over the zero-shot baseline. This result suggests that the category definitions alone carry sufficient signal for this task, and that in-context demonstrations, even when carefully constructed, may introduce a distributional bias that counteracts their informative value. The implications of this finding are discussed in Chapter 6.

5.6 Cross-Experiment Comparison

[To be completed: summary table comparing all four experimental scenarios across key metrics. Discussion of which approach performs best overall and which performs best per ambiguity type.]

Chapter 6

Discussion and Conclusion

6.1 Interpretation of Results

[To be completed: high-level reading of the experimental findings, unexpected patterns, and what the results suggest about the relationship between ambiguity and classification performance.]

6.2 Answers to Research Questions

6.2.1 RQ1: Does ambiguity negatively affect BERT classification performance?

[To be completed.]

6.2.2 RQ2: Can LLM disambiguation improve BERT classification accuracy?

[To be completed.]

6.2.3 RQ3: Can LLMs match or exceed BERT as direct classifiers?

RQ3 evaluates whether a general-purpose LLM can serve as a standalone classifier, replacing the fine-tuned BERT pipeline entirely. Three GPT-family models of varying capability ($GPT_{4.1\text{-nano}}$, $GPT_{5.4\text{-mini}}$, $GPT_{5.5}$) are tested in both zero-shot and few-shot configurations on the full 2,166-sentence dataset. Performance is compared against the BERT baseline using the same multi-label metrics (exact-match accuracy, Hamming loss, Jaccard score, micro/macro/per-label F1) and binary requirement F1-score.

[To be completed: results and interpretation.]

6.3 Threats to Validity and Limitations

[To be completed: dataset size, single-annotator labeling, LLM non-determinism, generalisability beyond cloud computing domain.]

6.4 Future Work

[To be completed: multi-annotator studies, other domains, fine-tuned LLM classifiers, automated ambiguity detection.]

6.5 Conclusion

[To be completed: brief summary of the thesis contributions and closing remarks.]

Bibliography

[1] Muneera Bano. "Addressing the Challenges of Requirements Ambiguity: A Review of Empirical Literature". In: *2015 IEEE Fifth International Workshop on Empirical Requirements Engineering (EmpiRE)*. Ottawa, ON, Canada: IEEE, 2015, pp. 21–24.

[2] Tom B. Brown et al. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems* 33 (2020).

[3] Emiliano Casalicchio and Alberto Cotumaccio. "AI-CRAS: AI-driven Cloud Service Requirement Analysis and Specification". In: *2015 IEEE Fifth International Workshop on Empirical Requirements Engineering (EmpiRE)*. Ottawa, ON, Canada: IEEE, Aug. 2015. DOI: 10.1109/EmpiRE.2015.7431303.

[4] Jacob Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: (2019). arXiv: 1810.04805.

[5] OpenAI. *GPT-4 Technical Report*. Tech. rep. OpenAI, 2023.

[6] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems* 30 (2017).

[7] Jason Wei et al. "Finetuned Language Models Are Zero-Shot Learners". In: *International Conference on Learning Representations* (2022).

Appendix A

Appendix

[Appendix content to be added.]

Acknowledgements

[Acknowledgements to be written.]

Claudio Giannini, Rome, March 2026